

{dev/talles}

Prompts para Desarrolladores

Por: Ricardo Cuellar

¿Qué es un Token?

Es la unidad mínima de texto que un modelo de IA procesa.

Puede ser:

- Una palabra completa
- Parte de una palabra
- Un signo
- Espacios

Depende mucho del modelo



¿Por qué importan los tokens?

Porque todo gira al rededor de ellos:

1. Costo
2. Rendimiento
3. Calidad





¿Qué es Context Window?

Es la cantidad máxima de tokens que el modelo puede “recordar” o procesar en una sola interacción.

- Prompt
- El contexto
- La respuesta del modelo

Ejemplo:

Context Window del modelo: 100,000 tokens

Envíamos: 80,000 tokens

Sobra para responder: 20,000 tokens



¿Cuál sería el problema?

Si se rebasa el contexto:

- Se recorta la información.
- Se pierde contexto importante
- El modelo empieza a olvidar



¿Qué son los Prompts?

Un prompt es una especificación de entrada que guía el comportamiento de un modelo para producir un resultado determinado.

prompt(Entrada + Instrucciones + Contexto) → Modelo → Salida



PROMPT



Ejemplo

Prompt débil:

“Explicame base de datos”

Prompt mejor definido:

“Explica qué es una base de datos relacional en 5 puntos, incluye un ejemplo en SQL y limita la respuesta a 150 palabras”



Nota importante

No debemos creer que entre más largo el prompt en automático dará mejores resultados. Debemos tomar en cuenta que un Prompt debe ser:

- Claro
- Específico
- Sin ambigüedad





Componentes de un prompt

- Objetivo claro
 - Lo que queremos lograr
- Contexto
 - Información relevante para mejorar la respuesta
- Restricciones
 - Longitud, formato, tono, etc.
- Formato de salida
 - JSON, lista, código, etc.



Roles en prompts

Rol: System

Es la “personalidad” o “contrato” con la IA. Es el de mayor jerarquía.

Rol: User

Lo que envía el usuario humano (texto, imágenes, documentos, etc).

Rol: Assistant

Las respuestas previas de la IA. Se usa para pre-cargar respuestas.



Estructura de un prompt

1. Dar un Rol.
2. Tarea
3. Contexto dinámico
4. Restricciones
5. Formato de salida





Ejemplo:

Eres un senior backend engineer experto en APIs REST

Tu tarea es diseñar un endpoint para crear usuarios

Contexto:

- nombre: {{name}}
- email: {{email}}

Restricciones:

- Usa buenas prácticas REST
- No incluyas explicaciones
- Sé conciso

Formato de salida:

Devuelve un JSON con:

```
{  
  "endpoint": string,  
  "method": string,  
  "description": string  
}
```



Diferencias:

Haz una API en Node.js



Eres un backend engineer senior.

Tu tarea es crear un endpoint REST en Node.js usando Express.

Contexto:

- recurso: usuarios
- acción: crear usuario

Restricciones:

- Usa validación con middleware
- Maneja errores correctamente
- No expliques nada

Formato de salida:

Devuelve solo código listo para producción.



Diferencias:

Explicame Docker



Explica Docker para un desarrollador backend con 2 años de experiencia.

Restricciones:

- Máximo 5 puntos
- Incluye 1 ejemplo práctico con docker run
- Evita definiciones teóricas largas

Formato:

Lista numerada



Diferencias:

Haz tests para esta
función:

{{código}}



Eres un QA engineer experto en testing.

Tu tarea es generar pruebas unitarias.

Contexto:

Función:

{{codigo}}

Restricciones:

- Usa Jest
- Cubre casos felices y errores
- Incluye edge cases

Formato:

Devuelve solo el archivo de test

Consejo:



No pidas y esperes la magia



Mejor planifica, pide con propósito y el resultado se te dará como esperas.



Gracias



www.devtalles.com



[/ricardocuellar](https://www.linkedin.com/company/ricardocuellar)



[@DevTalles](https://twitter.com/DevTalles)